

Chapter 9: Namespaces and Error Handling

In this lesson we are going to take a look at namespaces and error handling. Namespaces refer to a collection of classes and/or other objects defined under one unique name. Say for example that you have a large team of programmers all working on the same project. Do you suppose it is possible that two or more of those teams use the exact same class name but in reality, are referring to two totally distinct real-world entities. What if the gang coding student registrations has a totally different set of properties and methods for their student class than the gang of coders working on students actually taking a course. Trust me, it happens, and my teams were relatively small. What do you suppose Microsoft has to deal with?

Net, namespaces allow for large teams to have differing definitions associated with the same class name.

Error handling is just that, managing bad data input by the user and avoiding throwing unexpected error messages at that same user. Here is an example, you ask the user to input the file name of a self-picture to be uploaded and stored to your application. They are certain they know the name and folder location of that file, so they simply type it in instead of browsing and clicking. They of course get it wrong. You have two options as a coder at this point. Blindly accept this erroneous input and try to upload a non-existent file or go out there first behind the scenes and check whether the file exists. Option one throws an error while option two gently goes back to the user with the browse option. Yeah, option two is better.

Here is the deal. I will teach you how to “catch” and “throw” an error because I know this stuff is on every certification test. The real-world solution is to avoid them in the first place by writing smarter code. Anticipate issues and check whether they exist before generating an error.

So, let’s jump to the inside and look at some namespace and error handling examples. Both as Microsoft would have you believe is correct and how real coders get it done properly.

We begin by creating a new console project named Chapter 9 namespaces and error handling. No, I am not going to show you an image of this process, you should have this down cold by now.

The example shown here (Figure 9.1) introduces the new keyword namespace. This keyword is always immediately followed by a solution-wide unique namespace identifier. In our example here we have two namespaces aptly named namespace1 and namespace2.

Please note that in the second namespace I have intentionally miscoded the frame get constructor so we can clearly see the use of the exact same class names with differing constructor definitions in the two namespaces.

Figure 9.1: Namespaces

```
namespace namespace1
{
    0 references
    class Person
    {
        private string fn, ln;
        0 references
        public string fname { get { return fn; } set { fn = value; } }
        0 references
        public string lname { get { return ln; } set { ln = value; } }
    }
}

namespace namespace2
{
    0 references
    class Person
    {
        private string fn, ln;
        0 references
        public string fname { get { return ln; } set { fn = value; } } // wrong!!
        0 references
        public string lname { get { return ln; } set { ln = value; } }
    }
}
```

In this next image (Figure 9.2) we see the calling routine and its associated output. I know this example is pretty simplistic, but it demonstrates the value of namespaces, i.e. the ability to have multiple copies of the exact same class name within a solution.

Figure 9.2: Calling Routine and Output

```
0 references
class Program
{
    0 references
    static void Main (string[] args)
    {
        namespace1.Person RP = new namespace1.Person();
        RP.fname = "Rick";
        RP.lname = "Phillips";
        Console.WriteLine(RP.fname + " " + RP.lname);

        namespace2.Person RP2 = new namespace2.Person();
        RP2.fname = "Rick";
        RP2.lname = "Phillips";
        Console.WriteLine(RP2.fname + " " + RP2.lname);
    }
}
```

Microsoft Visual Studio Debug Console

Rick Phillips
Phillips Phillips

So how do you organize all these classes and namespaces in a large application? Good question and one which Microsoft has done a good job of answering. I first go up to the top menu and open the Project menu option. The very first sub-menu option is Add Class, go ahead and select that option. (Figure 9.3) From this next screen I select C# Class and Add and give my file a name and all my subsequently defined classes are neatly stashed away in a single file. By the way, if this were really a production system I would move any previously defined classes to this file as well. (Figure 9.4)

Do you suppose every team gets their own namespace and class definition file in a real solution? Yes, of course.

Figure 9.3: Project / Add Class

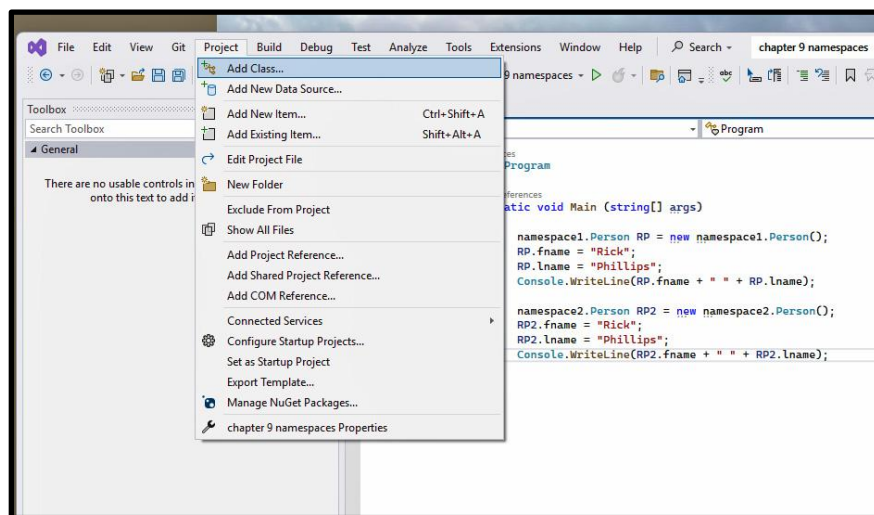
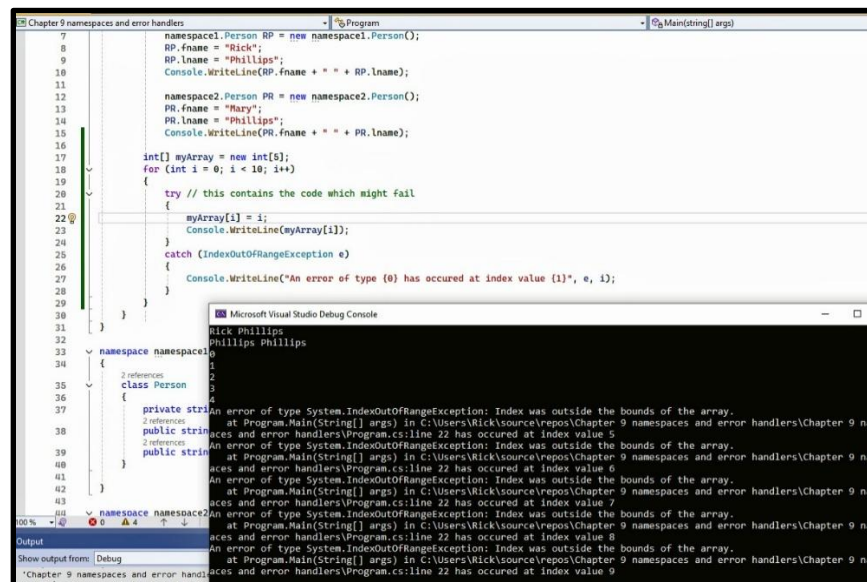


Figure 9.4: Add Class File Definition

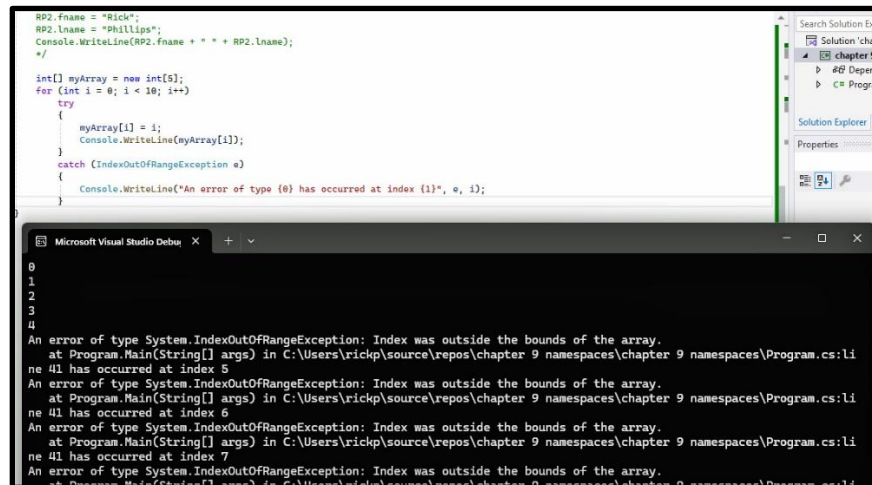


Okay, now we can move onto error handling. Here is the deal with error handling – avoid it if at all possible. If you know you need to open an external file and that file may not exist, then check for its existence before you try to open it. If you are looping through all the elements in a collection then don't use a specific number as your termination point, use the Length property of that collection. That all being said, those of you taking a certification test have to know this stuff because it will almost certainly be on there, so here we go. If you are not taking a certification test, feel free to skip these next couple of examples. There are better ways to handle this situation as we will show below.

Okay, if you are taking a test, I'll show you how not to write code. In this example I have added a try/catch system-defined exception handler (Figure 9.5). The try portion of the code is executed if valid. The catch portion of the code is executed if the specified exception occurs. In our case an `IndexOutOfRangeException`.

Please note that this example is known as a system-defined exception handler because the `IndexOutOfRangeException` is provided to you by Microsoft. In contrast, if you as the programmer define the exception as in our next example it is known as a user-defined exception. Personally, I don't think either should ever have to be employed and as proof of that assertion check out this alternative to the previous example. (Figure 9.5a) Does this guy ever throw an index out of range error? No. By the way, intellisense automatically gave me this option when I typed in the for keyword. Obviously the coders of C# intellisense know more than the test writers.

Figure 9.5: Try/Catch Exception Handler



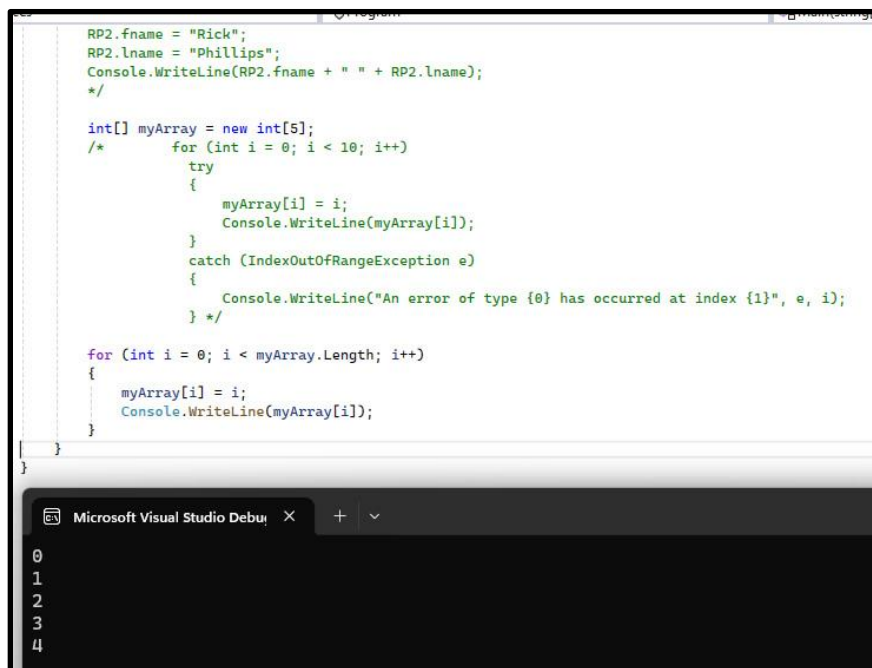
```
RP2.fname = "Rick";
RP2.lname = "Phillips";
Console.WriteLine(RP2.fname + " " + RP2.lname);
*/

int[] myArray = new int[5];
for (int i = 0; i < 10; i++)
{
    try
    {
        myArray[i] = i;
        Console.WriteLine(myArray[i]);
    }
    catch (IndexOutOfRangeException e)
    {
        Console.WriteLine("An error of type {0} has occurred at index {1}", e, i);
    }
}
```

Microsoft Visual Studio Debug Console Output:

```
0
1
2
3
4
An error of type System.IndexOutOfRangeException: Index was outside the bounds of the array.
at Program.Main(String[] args) in C:\Users\rickp\source\repos\chapter 9 namespaces\chapter 9 namespaces\Program.cs:li
ne 41 has occurred at index 5
An error of type System.IndexOutOfRangeException: Index was outside the bounds of the array.
at Program.Main(String[] args) in C:\Users\rickp\source\repos\chapter 9 namespaces\chapter 9 namespaces\Program.cs:li
ne 41 has occurred at index 6
An error of type System.IndexOutOfRangeException: Index was outside the bounds of the array.
at Program.Main(String[] args) in C:\Users\rickp\source\repos\chapter 9 namespaces\chapter 9 namespaces\Program.cs:li
ne 41 has occurred at index 7
An error of type System.IndexOutOfRangeException: Index was outside the bounds of the array.
```

Figure 9.5A: Better Alternative to IndexOutOfRangeException



```
RP2.fname = "Rick";
RP2.lname = "Phillips";
Console.WriteLine(RP2.fname + " " + RP2.lname);
*/

int[] myArray = new int[5];
/*
    for (int i = 0; i < 10; i++)
    {
        try
        {
            myArray[i] = i;
            Console.WriteLine(myArray[i]);
        }
        catch (IndexOutOfRangeException e)
        {
            Console.WriteLine("An error of type {0} has occurred at index {1}", e, i);
        }
    }
*/

for (int i = 0; i < myArray.Length; i++)
{
    myArray[i] = i;
    Console.WriteLine(myArray[i]);
}
}
```

Microsoft Visual Studio Debug Console Output:

```
0
1
2
3
4
```

In our next example we introduce the NoName user-defined exception. (Figure 9.6) Please note that our exception is inheriting from the Exception base class and contains three constructor overload options.

Figure 9.6: User-Defined Exception

```
namespace namespace1
{
    0 references
    class Person
    {
        private string fn, ln;
        0 references
        public string fname { get { return fn; } set { fn = value; } }
        0 references
        public string lname { get { return ln; } set { ln = value; } }
    }

    3 references
    public class NoNameException
    {
        0 references
        public NoName() { }
        0 references
        public NoName(string msg) : base(msg) { }

        0 references
        public NoName(string msg, Exception innerException) : base(msg, innerException) { }
    }
}
```

This image shows the calling routine for our user-defined exception handler. (Figure 9.7) The try section would normally contain code which processed properly formatted first and last names while the catch code throws an instance of our user-defined exception passing the constructor a message.

I know this looks like a cop out at this point just writing this error message but soon we will learn about subroutines which are independent modules of code which perform a single, independent function. These subroutines accept both input and output parameters. This example would be part of a subroutine which accepts some inputs and outputs a coded error message parameter if the throw part of the code were executed. The calling routine, which would obviously interface with the user, would then go back to the user indicating the absence of required fields.

By the way, if you go to the top of your new code you will see that intellisense has automatically added two new namespaces for you. That's because you referenced some classes not yet defined within the current namespace and intellisense went out and referenced the appropriate namespaces on your behalf. It will do this for your user-defined classes too if they are defined within a *.cs file as we learned earlier in this chapter.

Figure 9.7: try/catch/throw keywords

```
namespace2.Person RP2 = new namespace2.Person();
RP2.fname = "Rick";
RP2.lname = "Phillips";
Console.WriteLine(RP2.fname + " " + RP2.lname);
*/

int[] myArray = new int[5];
/*    for (int i = 0; i < 10; i++)
    {
        try
        {
            myArray[i] = i;
            Console.WriteLine(myArray[i]);
        }
        catch (IndexOutOfRangeException e)
        {
            Console.WriteLine("An error of type {0} has occurred at index {1}", e, i);
        }
    } */

for (int i = 0; i < myArray.Length; i++)
{
    myArray[i] = i;
    Console.WriteLine(myArray[i]);
}
```

Okay, that concludes our very brief discussion of namespaces and error handlers, let's jump to the outside and review what we learned.

So why are namespaces cool? Easy one, they are cool because they allow large teams of programmers to work independently because they allow for multiple definitions of the same class name. To pull a namespace into your current application just use the keyword using and the name of the namespace at the top of your code.

Rick is not a big fan of error handlers. Why not just check for a possible error before implementing the possible exception? Anyway, we know Microsoft likes them and they write the certification exams, so we covered both system-defined and user-defined exceptions in the current lesson. No problem.

In our next lesson we will cover file I/O, that is file input and output. Do you suppose we are going to learn how to check for the existence of a file before we try to open or upload it? Of course! It will be great fun.